

Note di C# - Stack, Memory & Classi

PARTE 1: STACK e Memory in C#

Architettura della Memoria

```
Stack "          • Stack Frame: Parameters / Return address / Local
variables

Heap '
    $
Global      • Global variables
    $
CODE        • Istruzioni del programma
```

Classificazione delle Variabili per Ubicazione in Memoria

Tipo di Variabile	Ubicazione
Value types (int, double, etc.)	Stack
Reference types (Array, oggetti)	Heap
Global variables	Global

Function Call - Meccanismi

Quando una funzione viene chiamata, avviene il seguente ordine di operazioni:

1. **push (parametri)** - I parametri vengono inseriti nello stack
2. **call (indirizzo)** - Viene salvato l'indirizzo di ritorno (Program Counter)
3. **push (variabili locali)** - Le variabili locali vengono allocate sullo stack

Registri Importanti Durante l'Esecuzione

- **PC/IP (Program Counter / Instruction Pointer)**
 - Contiene l'indirizzo della prossima istruzione da eseguire

- Salva la posizione del programma dove la funzione è stata chiamata prima della function call
- **BP (Base Pointer)**
 - Contiene l'indirizzo dello slot relativo allo stack frame corrente
- **SP (Stack Pointer)**
 - Contiene l'indirizzo del prossimo slot libero disponibile dello stack
- **Pop e Return**
 - Rimuovono lo stack frame corrente
 - Ripristinano i registri SP e BP al loro stato precedente

Best Practices - Memory Management

Regola per Value Types

Non creare local variable di tipo Value types di una funzione con dimensione > 64 bytes

Usa una reference type (array o classe) per dati più grandi

Problemi delle Global Variables

- Accessibili da tutte le funzioni del programma
- Difficile capire nel codice chi le modifica e quando
- Aumentano l'accoppiamento del codice

Stack vs Heap

Stack

- Ordinato con logica **LIFO** (Last In, First Out)
- Allocazione e deallocazione automatica e veloce
- Thread-safe per natura

Heap

- **NON ordinato**
- Gestito dall'**Heap Manager**
 - Gestisce l'allocazione/deallocazione
 - Mantiene traccia di quali blocchi di memoria sono liberi/occupati
- **Problema principale: HEAP FRAGMENTATION**

- Quando si deallocano oggetti, rimangono "buchi" di memoria frammentati

Garbage Collector (GC) in C#

Il Problema: Heap Fragmentation

Quando si allocano e deallocano oggetti sullo heap, si creano frammenti di memoria non contigua, rendendo inefficiente l'uso della memoria.

La Soluzione: Garbage Collector

- Si occupa di liberare la memoria non più utilizzata
- **Lavora in background periodicamente**
- Non blocca il programma principale

Come sa quali blocchi liberare? Reference Counting

*Ogni oggetto (**roots**) ha un **contatore** che tiene traccia di quante variabili fanno riferimento a quell'oggetto:*

***Se il contatore scende a 0** 'L'oggetto non è più utilizzato e può essere deallocato*

Ottimizzazione Finale

- Una volta liberata la memoria, i vari buchi liberi vengono **riuniti eliminando la frammentazione**
- **Grazie a ciò, in .NET l'allocazione della memoria è O(1)**

Esempio Pratico: Riduzione della GC Pressure

HIGH GC Pressure (Elevata)

```
// Crea un nuovo oggetto List ad ogni iterazione
for (...)
{
    List<int> vals = new();
    vals.Add(...);
}
```

Problema: Ad ogni iterazione viene allocato un nuovo oggetto sullo heap, aumentando il carico del Garbage Collector.

REDUCED GC Pressure (Ridotta)

```
// Riutilizza la stessa List, pulendola ad ogni iterazione
List<int> vals = new();
for (...)
{
    vals.Clear();
    vals.Add(...);
}
```

Soluzione: Alloca l'oggetto una sola volta, poi lo riutilizza pulendolo ad ogni iterazione. Il GC ha meno lavoro da fare.

PARTE 2: CLASSI in C#

Differenza tra Struct e Class

Aspetto	Struct (Value Type)	Class (Reference Type)
Tipo	Value Type	Reference Type
Memoria	Stack	Heap
Copia	Copia per valore	Copia per riferimento
Ideale per	Piccoli dati immutabili	Oggetti complessi

NOTA: Una classe è di tipo **reference type**, una struct è di tipo **value type**

Componenti di una Classe

1. Constructors (Costruttori)

- Il nome della funzione è **lo stesso della classe**
- **Non ha tipo di ritorno**
- Può essere **overloaded** (più di uno)
- Esegue l'inizializzazione dell'oggetto

```
class Rectangle {
    // Costruttore con parametri
}
```

```

public Rectangle (double l, double w) {
    Length = l;
    Width = w;
}

// Costruttore senza parametri (default)
public Rectangle() {
}
}

```

2. Fields (Campi)

- Sono **variabili della classe**
- Di default sono **private**
- Possono essere rese **public** (sconsigliato)
- **Convenzione di naming:** Prefisso `_` per i field privati

```

double _length; // Campo privato
double _width;  // Campo privato

```

3. Properties (Proprietà)

- Implementati come **metodi della classe**
- Ma hanno la **sintassi di un field** (senza parentesi)
- **get:** Consente di **leggere** il valore della proprietà
- **set:** Consente di **scrivere** il valore della proprietà
- **IMPORTANTE:** Mantengono la **consistenza dei dati**

```

public double Length {
    get => _length;
    set {
        if (value < 0)
            throw new Exception("Negative length not allowed");
        _length = value;
        // Invalidare i dati calcolati quando cambia Length
        _area = -1;
        _perimeter = double.NaN;
    }
}

```

4. Methods (Metodi)

- Sono **funzioni della classe**
- Di default sono **private**
- Possono essere rese **public**
- **override**: Sovrascrive il metodo della classe base

```
// Override del metodo ToString() della classe base Object
public override string ToString()
=> $"Rectangle {Length} x {Width} Area:{Area} Perimeter:{Perimeter}";
```

Avanzato: Lazy Evaluation & Sentinel Values

Cos'è la Lazy Evaluation?

Calcolare area e perimetro **SOLO** quando richiesti, non subito quando Length o Width cambiano.

Vantaggi:

- Riduce il lavoro computazionale
- Ottimizza le performance
- Mantiene la consistenza dei dati

Sentinel Values - Valori Speciali

Un **sentinel value** è un valore speciale che indica lo stato di una variabile:

Sentinel Value	Significato
-1	Valore non calcolato (usabile solo per numeri positivi garantiti)
double.NaN	Not a Number (valore non numerico)
null	Per reference types (array, stringhe, oggetti)

Esempio Pratico nel Codice

```
// Fields con inizializzazione a sentinel values
double _area = -1;           // -1 significa "non calcolato"
```

```

double _perimeter = double.NaN; // NaN significa "non calcolato"

// Proprietà Area con Lazy Evaluation
public double Area {
    get {
        if (_area < 0) // Se non è stato calcolato
            _area = _length * _width; // Calcola
        return _area;
    }
}

// Proprietà Perimeter con Lazy Evaluation
public double Perimeter {
    get {
        if (double.IsNaN(_perimeter)) // Se non è stato calcolato
            _perimeter = 2 * (_length + _width); // Calcola
        return _perimeter;
    }
}

// Nel setter, invalidare i sentinel values
public double Length {
    set {
        if (value < 0)
            throw new Exception("Negative length not allowed");
        _length = value;
        _area = -1; // Resetta a sentinel value
        _perimeter = double.NaN; // Resetta a sentinel value
    }
}

```

Alternative ai Sentinel Values

Invece di usare sentinel values, potresti usare:

1. **Flag separato** di tipo `bool`
2. **Nullable types** per reference types

Note Importanti

Inizializzazione di Default

Tutte le variabili private di una classe sono **inizializzate automaticamente** al loro valore di default:

- **Numeri** (int, double, float): `0`
- **Booleani**: `false`

- **Reference types:** null

Consistenza dei Dati - CRITICO

Quando una proprietà cambia:

- NON assumere che i dati calcolati siano ancora validi
- **Invalidare i valori calcolati** usando sentinel values
- Lasciar riprendere al Lazy Evaluation il ricalcolo al prossimo accesso

Esempio di Codice Completo

```
Rectangle r1 = new(10, 5);
r1.Length = 50;
r1.Width = 25;
Console.WriteLine($"Area: {r1.Area}"); // Area e Perimeter vengono
calcolati ORA

Rectangle r2 = new(10, 5);
Console.WriteLine(r2); // Output: Rectangle 10 x 5 Area:50 Perimeter:30
r2.Length = 100;
Console.WriteLine(r2); // Output: Rectangle 100 x 5 Area:500 Perime-
ter:210
Rectangle r3 = new(); // Usa il costruttore default
Console.WriteLine(r3); // Output: Rectangle 0 x 0 Area:0 Perimeter:0
```

Riepilogo Finale

Concetto	Descrizione
Stack	LIFO, veloce, value types, automatico
Heap	Non ordinato, reference types, gestito da GC
GC	Libera memoria non usata, usa Reference Counting
Classe	Reference type, contiene fields/properties/methods
Constructor	Inizializza l'oggetto
Properties	Accesso controllato ai dati con get/set

Lazy Evaluation	Calcola solo quando necessario
Sentinel Values	Indica lo stato speciale di una variabile